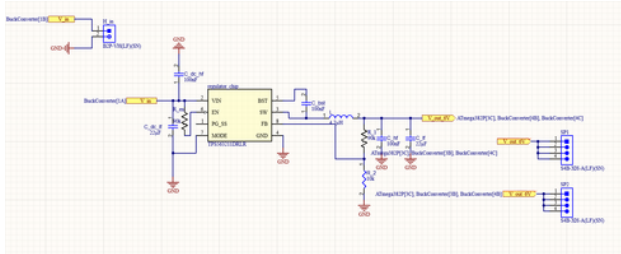
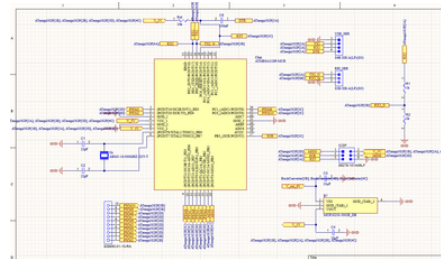


SERVO CONTROL BOARD - DATA SCIENCE AND INFORMATICS LAB

Buck Converter schematic



MCU integration schematic + LDO



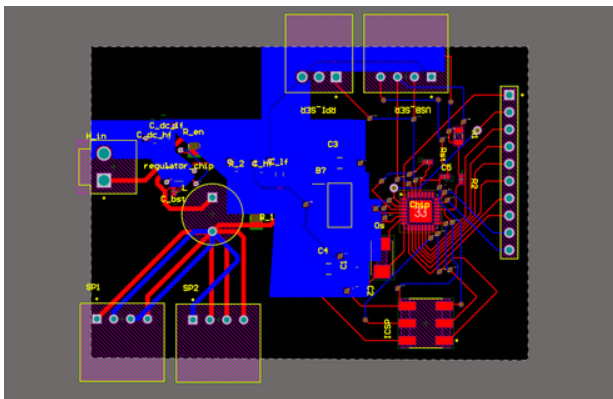
Situation

- Needed a centralized power, control, and communication system to work as a URDF robot custom end-effector
- Developed a control board with an in-built **voltage regulation, programming capabilities**, and **UART** communication

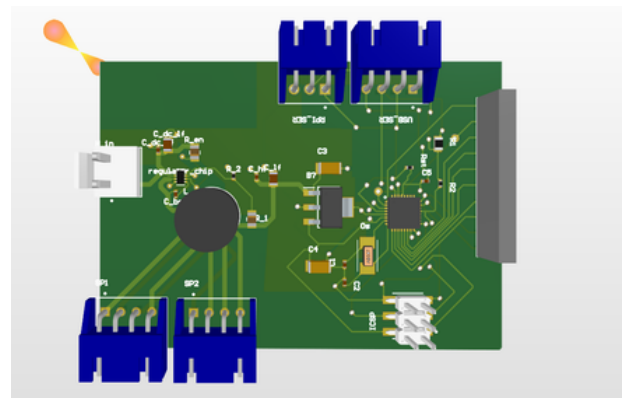
Method

- Designed all schematics in **Altium Designer**, referencing IC **datasheets** for circuit design, leading to a final **PCB design**
- Implemented **ATmega328P MCU** due to extensive community documentation and support, A **6V 3A switching buck converter** (TPS563211) to handle the peak stall currents of ~2A, **5V LDO linear voltage regulator** (MCP1815S) to safely power the ATmega

2D Altium PCB View



3D Altium PCB View



Results

- **Iterated** through multiple PCB designs until all features were functional
- Implemented **Root Cause Analysis** principles to diagnose PCB issues, systematically tracing ideal behavior and matching with actual behavior, utilizing a **Digital Multimeter**
- Developed a final working **2-layer** PCB with all features fully functional

Features

- **Voltage regulation** system utilizing a buck converter and a linear voltage regulator to safely power servos and the ATmega
- Enabled **UART** communication for MCU for firmware uploading and Rpi communication, adding a **resistive voltage** divider on TX Line, stepping down ATmega's 5V output to 3.3V, protecting the Raspberry Pi's GPIO from overvoltage allowing for safe communication between ATmega and Rpi
- **ICSP** Header for initial **bootloader** burning without the need for a pre-flashed chip

MICROMOUSE - UGA IEEE

International MicroMouse Competition



Navigation Feedback Loop Code

```
119 //Get information from all 12 sensors
120 for (int i = 0; i < 12; i++)
121 {
122     //If sensors are pre-set configured to follow the maze config
123     if (i == 0 || i == 11)
124     {
125         //wall exists in direction
126         if (downcast<int>(readSensor(i)) < 1000)
127         {
128             //Update maze representation
129             updateMaze(maze_location, true_dir, maze_grid, rep_wall, i, rep_x, y, rep_y);
130             //Update location
131             updateLocation(true_dir, grid, rep_location, heading, i, rep_x, y, rep_y);
132             //walls are defaulted to not exist, so no further code is necessary
133         }
134     }
135     //Find where the bot should move relatively
136     int dir = findPath(grid, rep_location);
137     //Travel in said direction
138     travelPath(dir, path, grid, dir, MOTOR_PWM);
139     //Update location accordingly
140     updateLocation(dir, path, i, rep_x, y, rep_y);
141     //Delay (delay: 500);
142 }
```

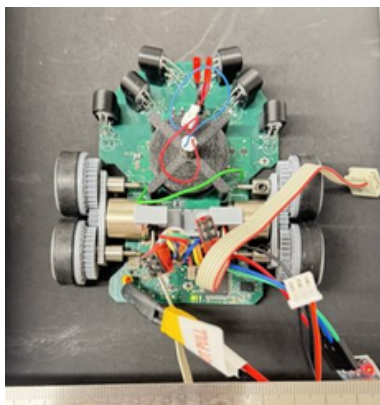
Method

- Utilized the **STM32 MCU** as the target hardware to develop all firmware
- Managed concurrent **I2C**, **UART PWM**, and **ADC peripherals** on STM32 using CubeMX and **HAL** drivers for IMU heading, Bluetooth telemetry, motor control, and wall detection
- Integrated all features into a **floodfill navigation firmware** in **embedded C**

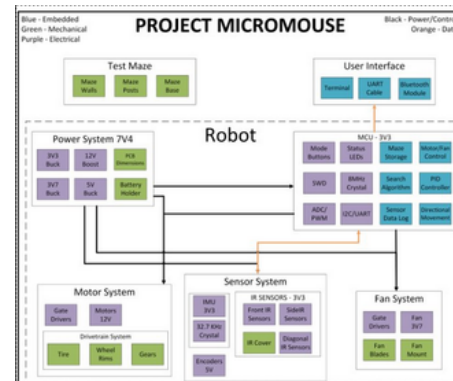
Situation

- Needed autonomous maze-solving firmware for a 16x16 competition maze with **real-time** wall detection and **dynamic path calculation**
- Required a navigation firmware that could run efficiently within **limited RAM** without blocking time-critical **peripheral communication**

Fully developed MicroMouse robot



Sensor Communication Diagram



Results

- Demonstrated **full maze solve** on test runs before competition, with firmware reliably managing all peripherals simultaneously under real conditions
- Implemented **live Bluetooth telemetry** to validate firmware state during development, confirming correct peripheral timing and navigation decisions in real time

Features

- Multi-peripheral firmware architecture** coordinating ADC, I2C, UART, and PWM simultaneously on a single STM32
- Real-time floodfill execution** within RAM and CPU constraints leveraging **DMA**-offloaded sensor reads, allowing for non-blocking simultaneous peripheral communication
- Wireless debug pipeline** through UART/Bluetooth for **live firmware state monitoring** during testing